# Artificial Intelligence

Beyond Classical Search

# Previous Search Algorithms

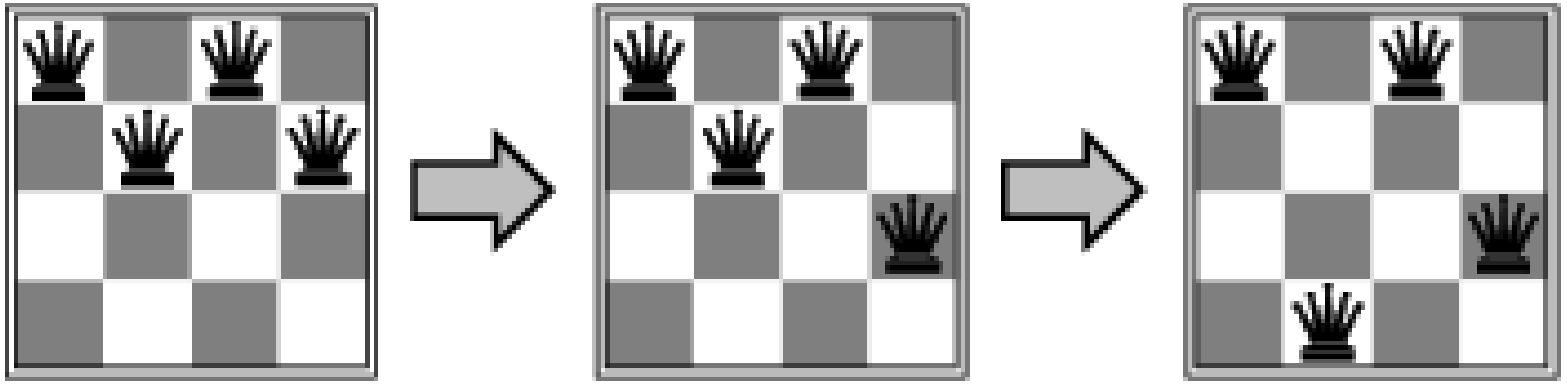
- Designed to explore search space systematically:
  - keep one or more paths in memory
  - record which have been explored and which have not
  - a path to goal represents the solution

# Local search algorithms

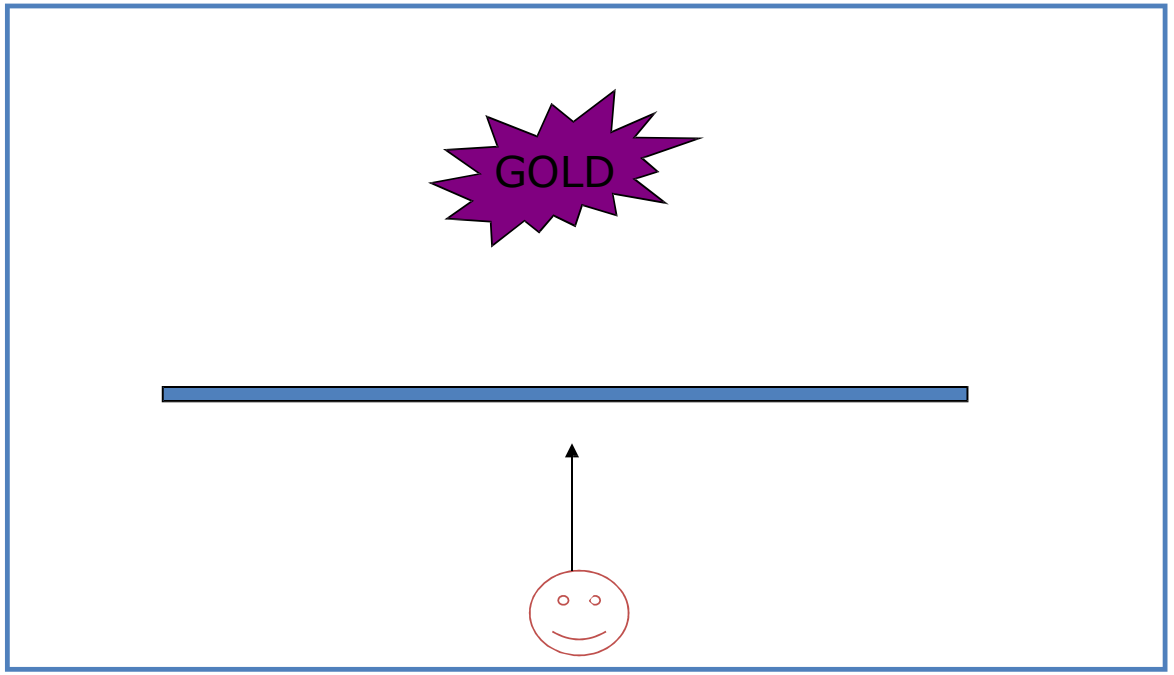
- In many optimization problems, the path to the goal is irrelevant; the goal state itself is the solution
- State space = set of "complete" configurations
- Find configuration satisfying constraints, e.g., n- queens
- In such cases, we can use local search algorithms
- keep a single "current" state, try to improve it

# Example: *n*-queens

- Put *n* queens on an *n* × *n* board with no two queens on the same row, column, or diagonal

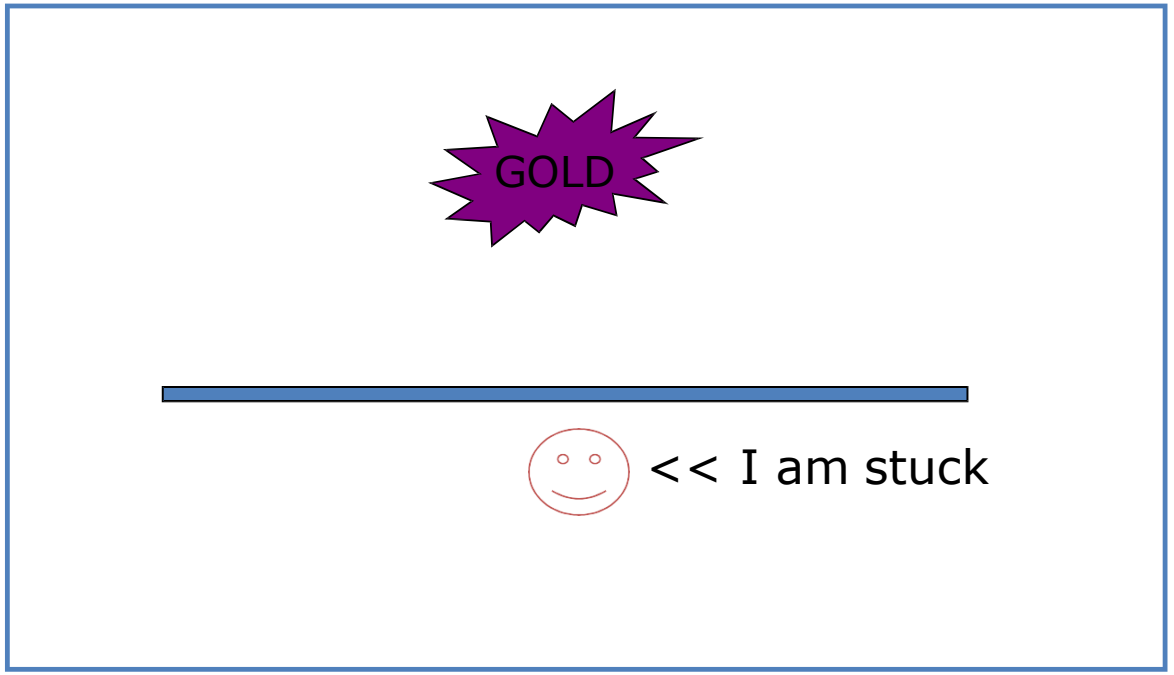


# Example: Initial State




Assume the objective function measures the straight-line distance

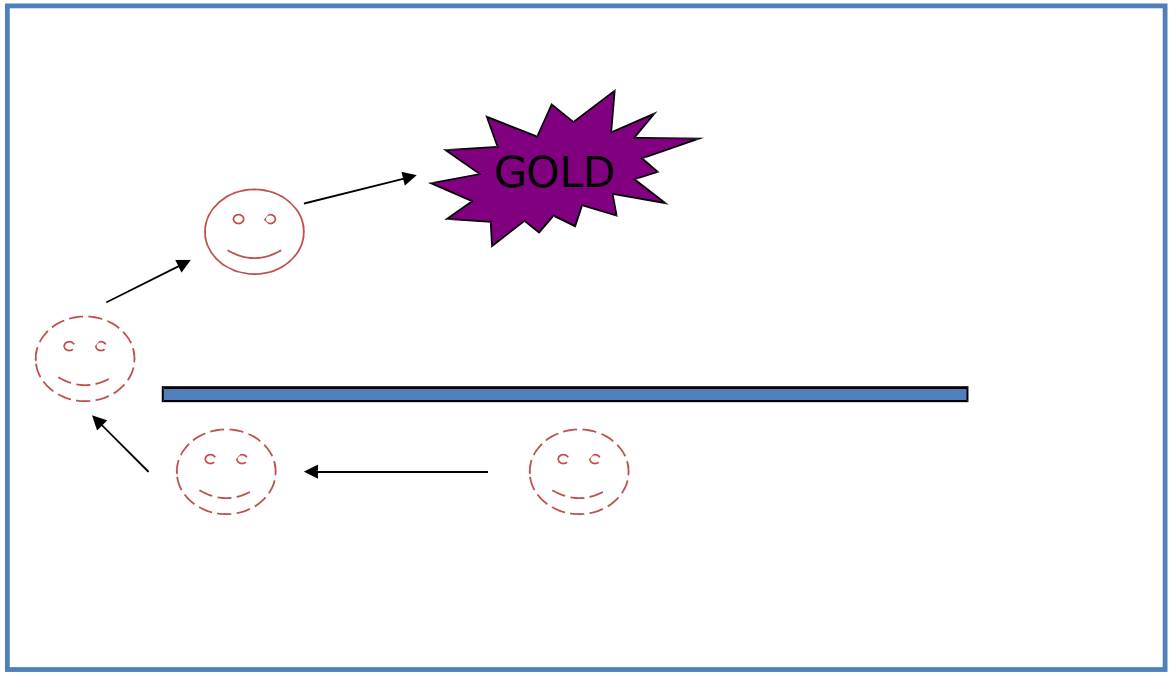
# Example: Local Minima




Assume the objective function measures the straight-line distance

# Example: A Plausible Solution

Making some “bad” choices is actually not that bad




Assume the objective function measure the straight-line distance

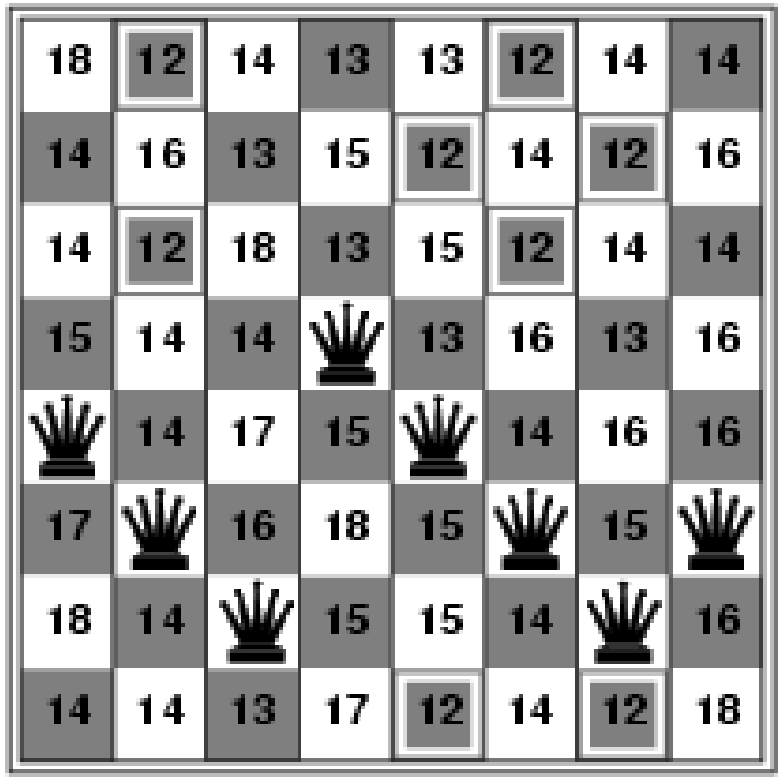
# Hill-climbing search

- "Like climbing Everest in thick fog with amnesia"

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
   inputs: problem, a problem
   local variables: current, a node
                    neighbor, a node

   current ← MAKE-NODE(INITIAL-STATE[problem])
   loop do
      neighbor ← a highest-valued successor of current
      if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
      current ← neighbor
```

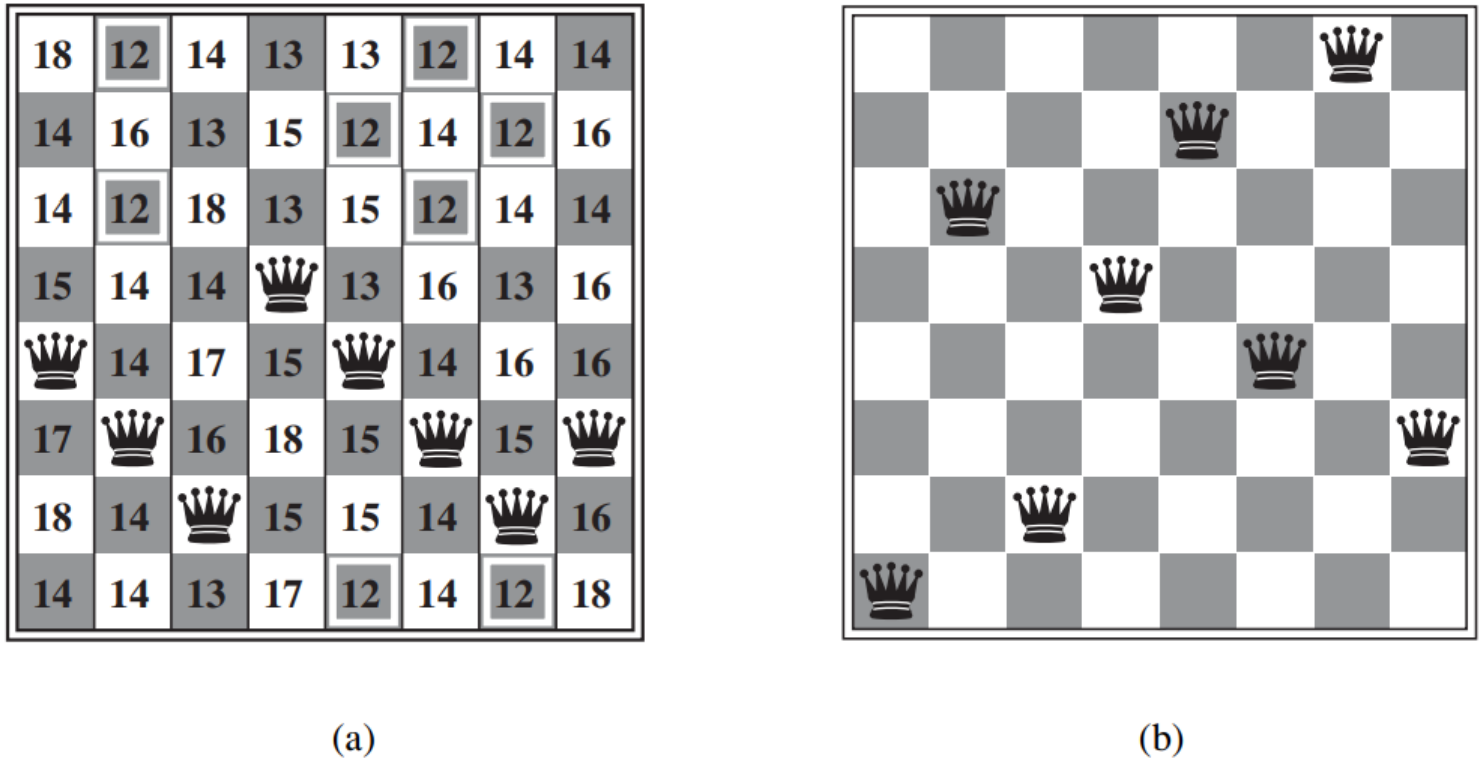
# Hill-climbing search: 8-queens problem




- *h* = number of pairs of queens that are attacking each other, either directly or indirectly

# Hill-climbing search: 8-queens problem

- It takes just five steps to reach the state in (b), which has h = 1 and is very nearly a solution



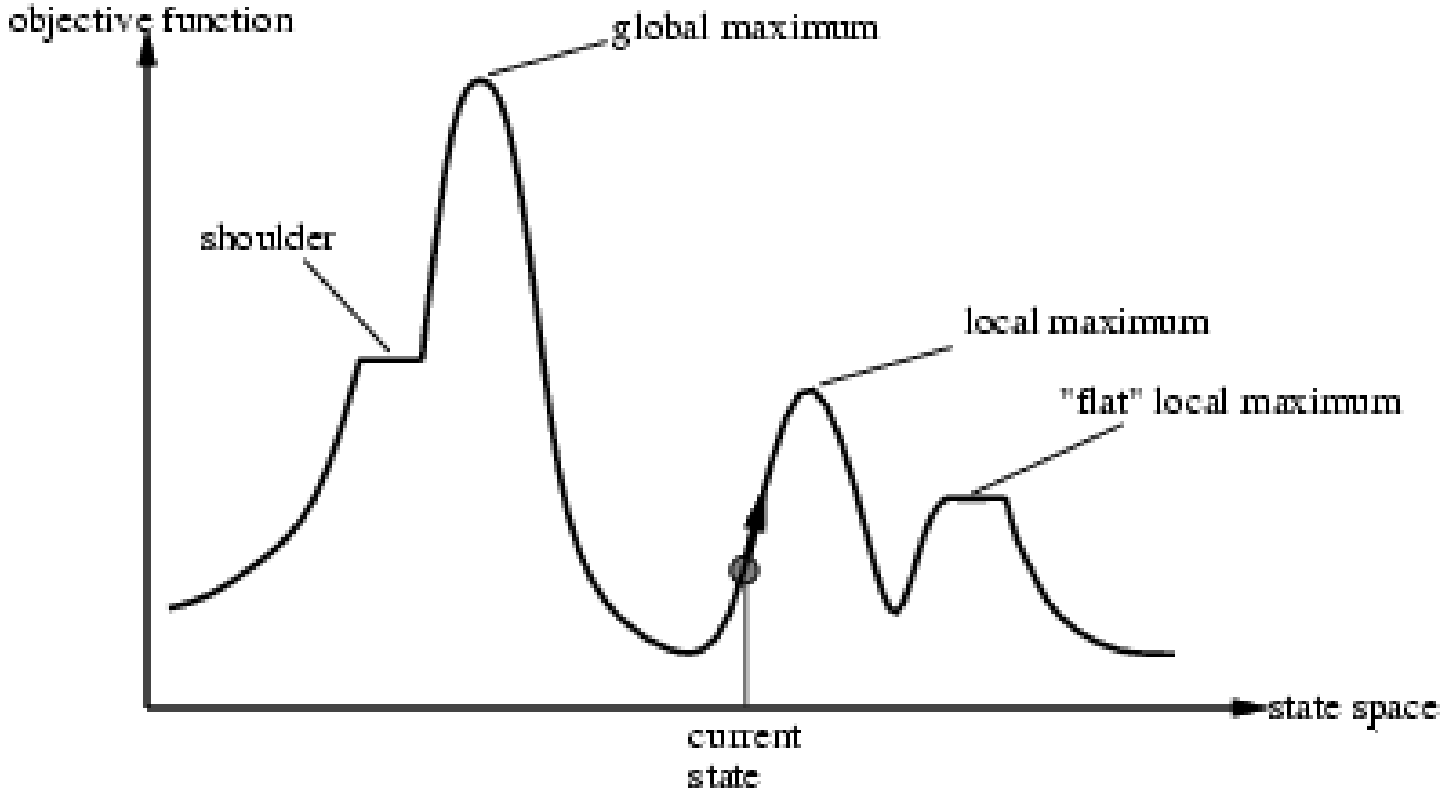

(a) (b)

**Figure 4.3** (a) An 8-queens state with heuristic cost estimate $h = 17$, showing the value of $h$ for each possible successor obtained by moving a queen within its column. The best moves are marked. (b) A local minimum in the 8-queens state space; the state has $h = 1$ but every successor has a higher cost.

# Hill-climbing search

- Problem: depending on initial state, can get stuck in local maxima

- Local maxima
- Ridges
- Plateaux



# Variants of Hill-climbing search

- Simple Hill Climbing
  - It examines the neighboring nodes one by one
  - selects the first neighboring node which optimizes the current cost as the next node
- Steepest Ascent
  - It first examines all the neighboring nodes
  - selects the node closest to the solution state as the next node.

# Variants of Hill-climbing search

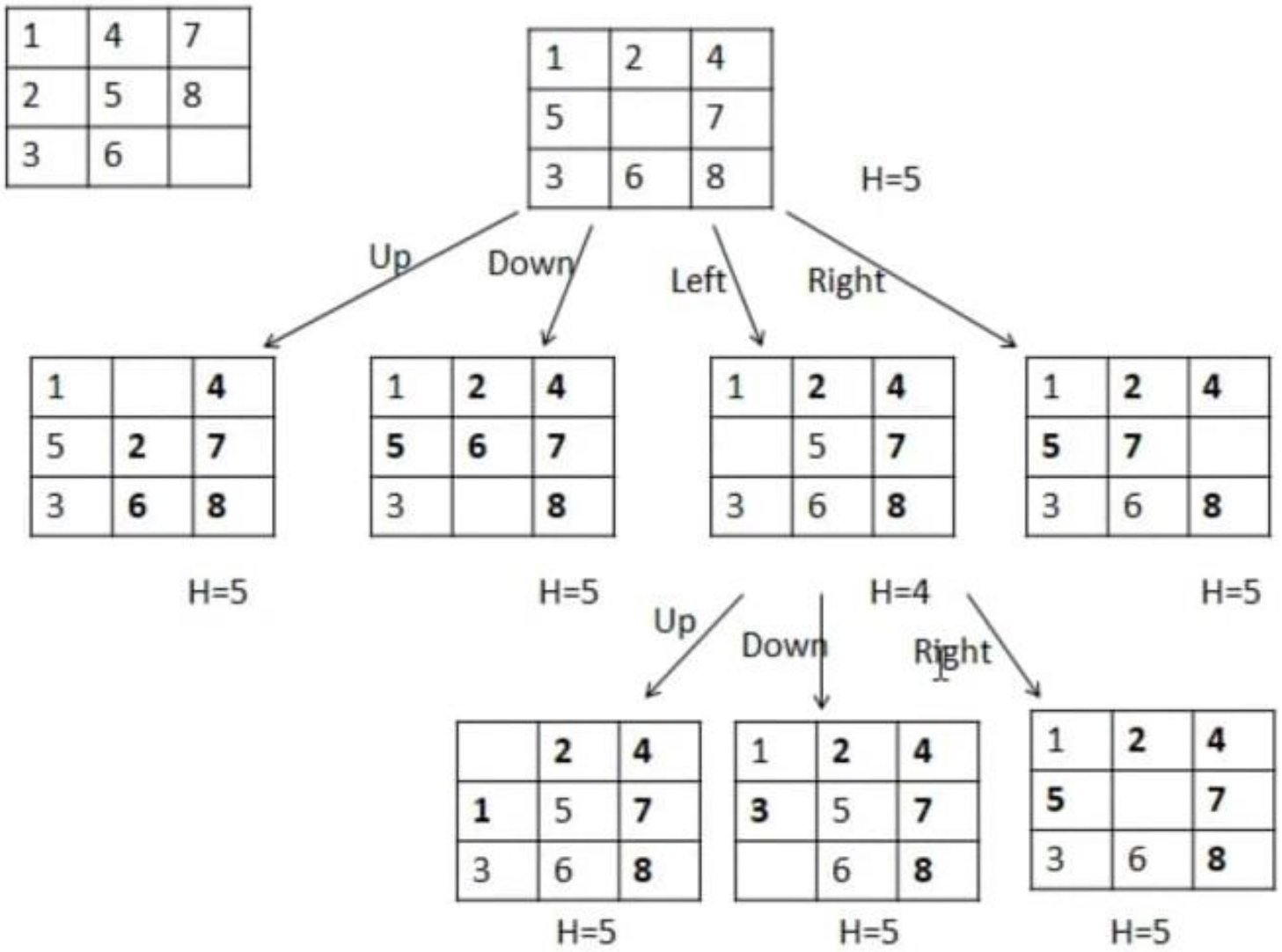
- Stochastic hill climbing:
  - chooses at random from among uphill moves
  - probability of selection can vary with the steepness of the uphill move
  - converges more slowly, but finds better solutions in some landscapes

*It does not examine all the neighboring nodes before deciding which node to select.*

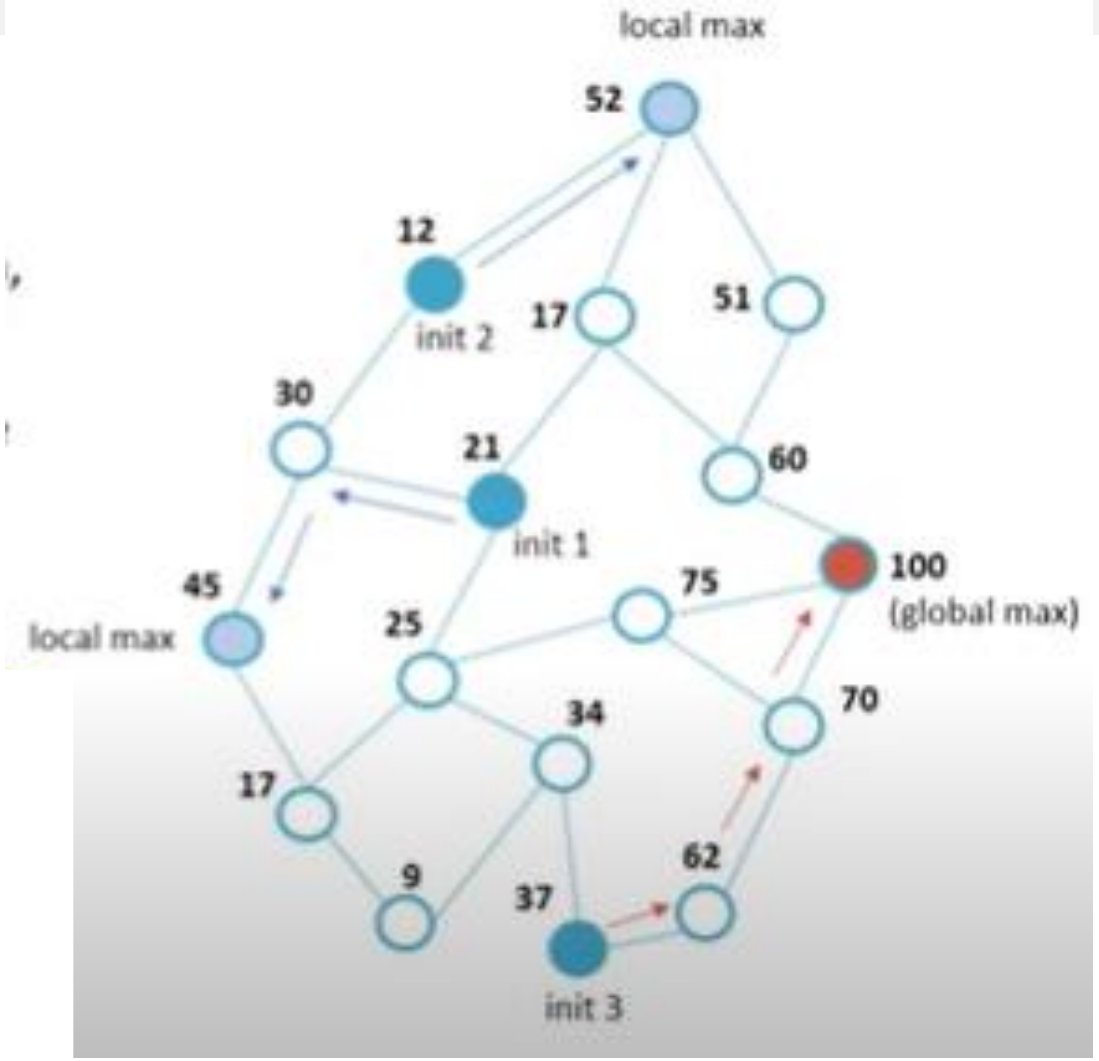
*It just selects a neighboring node at random and decides (based on the amount of improvement in that neighbor) whether to move to that neighbor or to examine another.*

# Variants of Hill-climbing search

- Random-restart hill climbing:
  - conducts a series of hill climbing searches from ***randomly generated initial states***, stops when a goal is found
  - It's trivially complete with probability approaching 1, because it will eventually generate a goal state as the initial state.


1 4 7
2 5 8
3 6
1 2 4
5 7
3 6 8
H=5
Up
Down
Left
Right
1 4
5 2 7
3 6 8
H=5
1 2 4
5 6 7
3 8
H=5
1 2 4
5 7
3 6 8
H=4
1 2 4
5 7
3 6 8
H=5
Up
Down
Right
2 4
1 5 7
3 6 8
H=5
1 2 4
3 5 7
6 8
H=5
1 2 4
5 7
3 6 8
H=5

# Random-Restart Hill Climbing



# Random-Restart Hill Climbing

- Assume each hill climbing search has a probability p of success, then the expected number of restarts required is 1/p

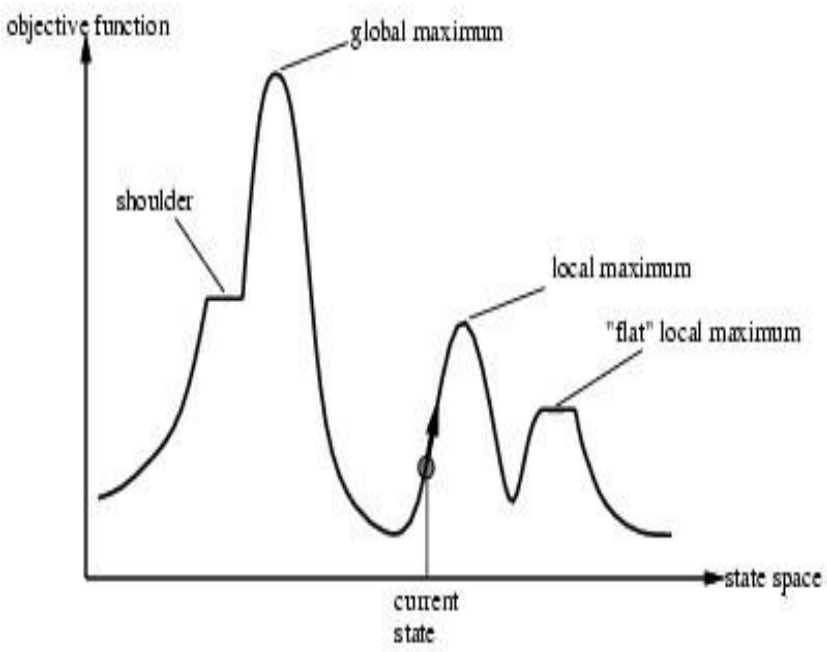
- For the 8-Queen problem with no sideways allowed, p = 14%, so we need roughly 7 iterations to find a goal (6 failures and 1 success)
  - if the algorithm is run multiple times, in the majority of cases, it will not find a solution in the first 6 iterations. However, in 1 out of 7 iterations, it will find a solution

# Random-Restart Hill Climbing

- Expected # of steps = cost_to_success + $\frac{1-p}{p}$ * cost_to_failure
  - **cost_to_success** is the cost (i.e., the number of steps or evaluations of the objective function) required to reach a goal state (i.e., a state with no further improvement possible)
  - $p$ is the probability of generating a state that leads to an improvement in the objective function.
  - **cost_to_failure** is the cost required to reach a local maximum, which is a state where no further improvement is possible through the available moves.

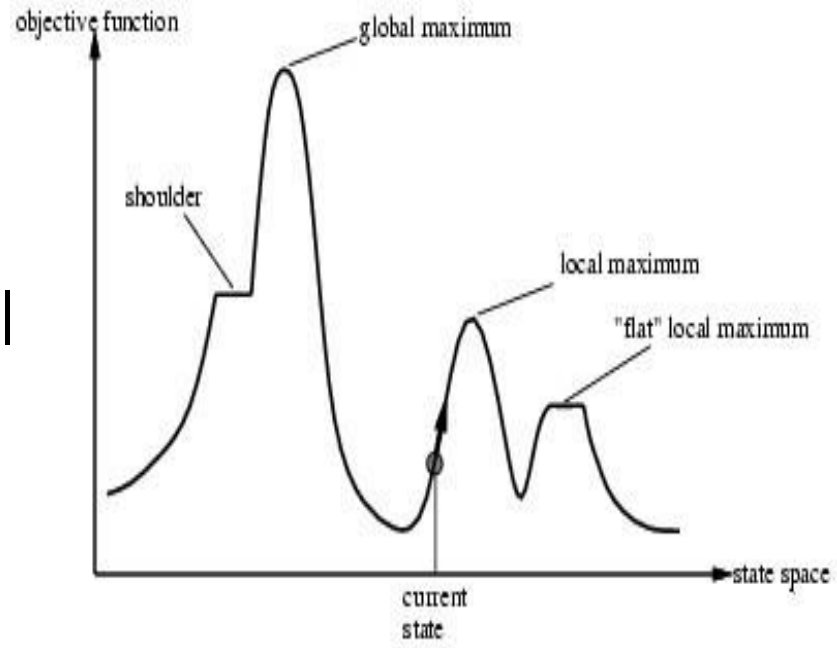
# Hill Climbing Search

- Random-restart hill climbing is very effective for n-queen problem if we allow sideway moves
- 3 million queens can be solved < 1 min
- Complete? Optimal?



# Hill Climbing Search

- Complete? Optimal?
- Hill climbing is sometimes called greedy local search
- Although greedy algorithms often perform well hill climbing gets stuck when:
  - Local maxima/minima
  - Ridges
  - Plateau (shoulder or flat local maxima/minima)



# Hill-Climbing Search

- NP-hard problems typically have an exponential number of local maxima/minima to get stuck on
- A hill climbing algorithm that never makes “downhill” (or “uphill”) moves is guaranteed to be incomplete
- A purely random walk – moving to a successor chosen uniformly at random – is complete, but extremely inefficient

# Hill-Climbing Search

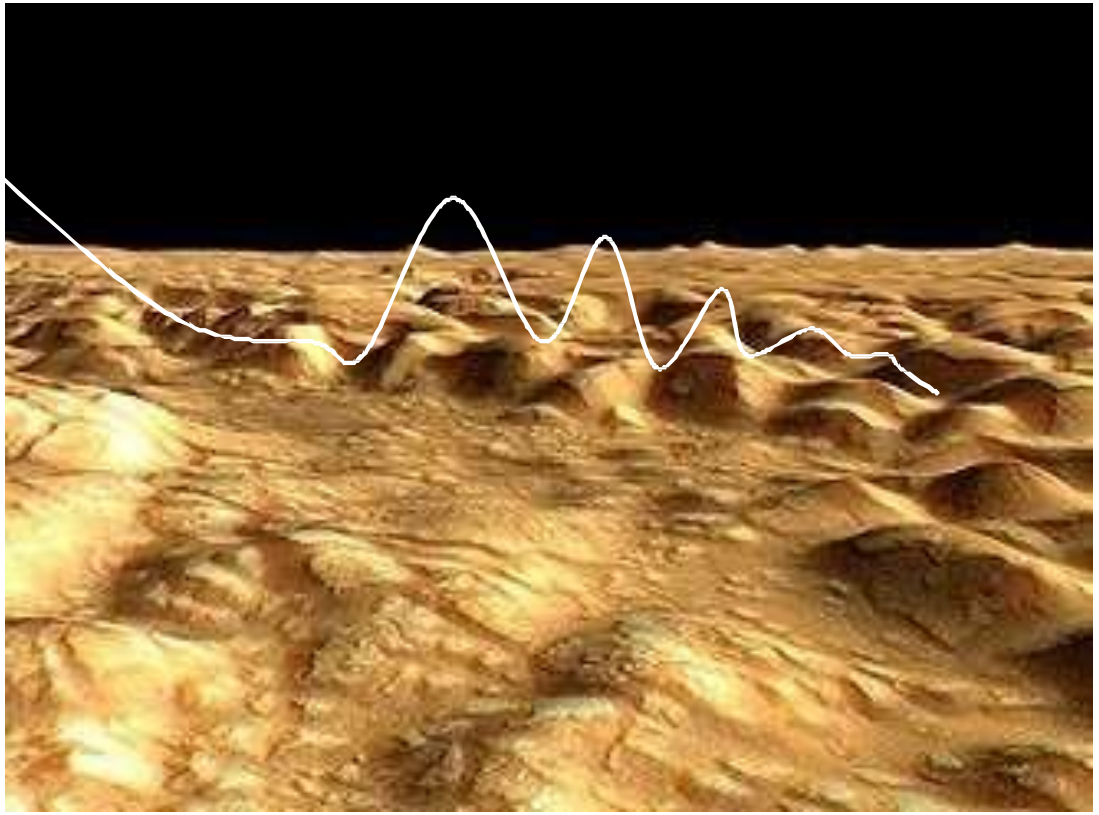
- What should we do?
- Annealing:
  - The process used to harden metals and glass by heating them to a high temperature and then gradually cooling them, thus allowing the material to reach a low-energy crystalline state

# Ping-Pong Ball Example

To explain simulated annealing, we switch our point of view from hill climbing to **gradient descent.**

**Gradient descent** is an optimization algorithm that is commonly used to **find the minimum of a function**. The algorithm works by iteratively adjusting the values of the variables in the function to minimize the value of the function.

The trick is to shake just hard enough to bounce the ball out of local minima but not hard enough to dislodge it from the global minimum



# Simulated annealing search

- Idea: escape local maxima by allowing some "bad" moves but gradually decrease their frequency

Simulated annealing is a probabilistic variation of Hill Climbing that allows the algorithm to occasionally accept worse moves in order to avoid getting stuck in local maxima.

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
   inputs: problem, a problem
           schedule, a mapping from time to "temperature"
   local variables: current, a node
                    next, a node
                    T, a "temperature" controlling prob. of downward steps

   current ← MAKE-NODE(INITIAL-STATE[problem])
   for t ← 1 to ∞ do
       T ← schedule[t]
       if T = 0 then return current
       next ← a randomly selected successor of current
       ΔE ← VALUE[next] – VALUE[current]
       if ΔE > 0 then current ← next
       else current ← next only with probability e^(Δ E/T)
```

# Properties of simulated annealing search

- Instead of picking the best move, however, it picks a random move
- If the move improves the situation, it is always accepted. Otherwise, the algorithm accepts the move with some probability less than 1.
- The probability decreases exponentially with the “badness” of the move—the amount $\Delta E$ (difference of next node – current node) by which the evaluation is worsened.
- The probability also decreases as the “temperature” T goes down: “bad” moves are more likely to be allowed at the start when T is high, and they become more unlikely as T decreases.
- If the schedule lowers T slowly enough, the algorithm will find a global optimum with probability approaching 1.

# Properties of simulated annealing search

- One can prove: If *T* decreases slowly enough, then simulated annealing search will find a global optimum with probability approaching 1.

- Widely used in VLSI layout, factory scheduling etc.